2014 IEEE 23rd Asian Test Symposium

Generator for Test Set Construction of SMGF in Reversible Circuit by Boolean Difference Method

Bappaditya Mondal[1], Dipak Kumar Kole[1], Debesh Kumar Das[2] and Hafizur Rahaman[1]
[1]Department of Information Technology
[1]Indian Institute of Engineering Science and Technology, Shibpur, India
[2]Department of Computer Science and Engineering
[2]Jadavpur University, Kolkata, India
{Email:bappa.arya@gmail.com, dipak.kole@gmail.com, debeshd@hotmail.com, rahaman_h@it.becs.ac.in}

***Abstract*—Reversible logic synthesis has received considerable attention in the light of advances recently made in quantum computation. Implementation of a reversible circuit is envisaged by deploying several special types of quantum gates, such as k-CNOT. Although the classical stuck-at fault model is widely used for testing conventional $CMOS$ circuits, new fault models, namely single missing-gate fault $(SMGF)$, repeated-gate fault (RGF), partial missing-gate fault $(PMGF)$, and multiple missing-gate faults $(MMGF)$, are likely to be more suitable for modeling defects in quantum k-CNOT gates. This work proposes an algorithm for deriving the test set for the detection of all single missing gate faults in a reversible circuit implemented with k-CNOT gates. Instead of deriving test set directly for the detection of missing gate faults, a Boolean generator is developed by Boolean difference method to derive the test set and to detect all the single missing gate faults of a reversible circuit. Experimental results on some benchmark circuits are also reported.**

Keywords*—Missing-gate faults, quantum computing, reversible logic, testable design, universal test set, Boolean difference.

I. Introduction

Reversible logic can be employed to design information lossless circuits. According to Landauer's principle [1], a certain amount of energy ($KTlog2\ Joules$) is dissipated as heat in the conventional logic computation due to the loss of every bit of information during the computation. Reversible logic may offer a viable solution of reducng heat dissipation in the conventional computing in future with newer technologies, e.g., spintronics. It provides a basis for the newly emerging paradigm of quantum computing [2], [3].

Reversible circuits [4], [5] have numerous applications to optical computing, cryptography, nanotechnology, DNA technology, and low-power $CMOS$ design [6], [7]. Since quantum gates or circuits satisfy no-cloning behavior, and are information lossless, they do not permit fan-out, and ought to have an equal number of inputs and outputs. An n-input, m-output Boolean function F is said to be reversible if and only if $m = n$, and F is one-to-one. A reversible combinational circuit should consist of only reversible gates, which themselves implement reversible functions; such gates need to be specially designed, e.g., by using controlled-not $(CNOT)$,Feynman [8],Fredkin [9] or Toffoli [10] gates. Recently, several researchers have studied the problem of fault modeling and testing of reversible logic circuits [11], [12], [13], [14], [15]. The Universal testability of reversible logic circuits implemented with k-CNOT gates under the stuck-at fault model (both single and multiple) has also been investigated in [16]. However, new fault models, namely, $SMGF$, RGF, $PMGF$, and $MMGF$ [17], [18], [19], [20], [21], have been found to be more suitable for modeling, defects in quantum k-CNOT gates [22], [23]. A k-CNOT based circuit can be implemented using trapped-ion technology, where the ions interact with laser pulses [23]. Determination of universal test set [24] for fault detection in reversible circuits has been studied for these fault models in [25], [22], [16]. The problem of detecting multiple missing-gate faults $(MMGFs)$ by integer linear programming has been investigated in [23]. It is shown earlier that in an (n, n) reversible circuit implemented with k-CNOT gates, addition of only one extra control line along with duplication of each k-CNOT gate yields an easily testable design, which admits a universal test set of size $(n + 1)$ that detects all $SMGFs$, $RGFs$, and $PMGFs$ [26]. A compact test vector generation algorithm for reversible circuit to achieve high (100%) fault coverage is presented in [27].

This work introduces a Boolean difference technique for generating test sets for reversible logic circuits. It shows how the generator of the test set is constructed to detect all the single missing gate faults for the entire circuit. The test set is derived from the Boolean expression only.

The paper is organized as follows. In section II, the preliminaries of reversible logic circuits are presented. The section III discusses the proposed fault detection method in a reversible logic circuit using Boolean expression. Experimental results are presented in section IV. Finally, section V concludes the paper.

II. PRELIMINARIES

A. Reversible gates:

A reversible gate is defined as $g(C, T)$ where C is called the set of control lines and T is the set of target lines with $C \cap T = \phi$. In the reversible gate, the control set C may be empty but T contains at least one target line. A set of reversible logic gate is used to build reversible circuits. The basic reversible gates are defined as follows:

1) 1-input/1-output NOT gate: It is a basic reversible gate. According to the definition of reversible gate, it has only one target line (T) but no control line (C) is used in this gate. It is realizing the expression $(X \rightarrow \bar{X})$. 1-input/1-output reversible NOT gate is shown in Fig.1.

1081-7735/14 $31.00 © 2014 IEEE
DOI 10.1109/ATS.2014.24

IEEE computer society

Fig. 1: Reversible NOT gate

2) 2-input/2-output Controlled NOT ($CNOT$) gate: According to the definition of reversible gate, 2-input/2-output Controlled NOT gate is canstructed by 1 control line and 1 target line. It is realizing the expression $(X_1, X_2) \rightarrow (X_1, X_1 \oplus X_2)$. The $CNOT$ gate is shown in Fig.2.

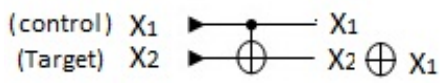

Fig. 2: $CNOT$ gate

3) 3-input/3-output Toffoli gate: The Toffoli gate has 2 control line and 1 target line. It is realizing the expression $(X_1, X_2, X_3) \rightarrow (X_1 \cdot X_2 \oplus X_3)$. The Toffoli gate is shown in Fig.3.

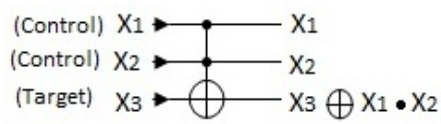

Fig. 3: Toffoli gate

A k-CNOT gate has k control inputs x_1, x_2,·····,x_k and one target input t. It maps the input vector x_1, x_2,·····, x_k, t to the output vector x_1, x_2,·····,x_k, $t \oplus x_1 \cdot x_2 \cdots x_k$. Thus, a k-CNOT gate has $(k+1)$ inputs and $(k+1)$ outputs; the first k outputs follow the respective inputs, and it inverts the target at the $(k+1)^{th}$ output if and only if all the k control inputs are 1.

B. Fault Model

A fault is a physical defect in a system [28]. In other words, a system has a fault if there is a physical difference between the correct system and the current system [29]. A fault model categorizes various physical faults within the circuit and also a fault model describes the behaviour of physical faults within the circuit. Several new fault models for k-CNOT based reversible circuits have been introduced earlier in [22], [23]. These are single missing gate fault ($SMGF$), repeated gate fault (RGF), partial missing-gate fault ($PMGF$), and the multiple missing-gate fault ($MMGF$). In this section, we briefly discuss the concept of $SMGF$.

Single missing-gate fault (SMGF): The $SMGF$ model corresponds to the case when one k-CNOT gate completely disappears from the circuit. In the presence of this fault, the $CNOT$ gate behaves as a simple wire connection, i.e., the pulse implementing the gate operation is short, missing, misaligned or mistuned [23]. Fig. 4 shows an $SMGF$ marked by the dotted box in the reversible circuit ham3 design1 benchmark, where the first 2-CNOT gate is missing. An $SMGF$ is detected by setting logic 1 value on all the control inputs of the gate, and any value either 0 or 1 on the target input as well as on the wires not connected to the gate. In the example of Fig. 4, if we apply $(a, b, c) = (0, 1, 1)$ at the input of the circuit, the normal output would be $(e, f, g) = (1, 0, 0)$, whereas, in the presence of the $SMGF$ fault marked by the dotted box, the output will be $(e, f, g) = (0, 1, 1)$. The number of possible $SMGFs$ is equal to the number of gates in the circuit.

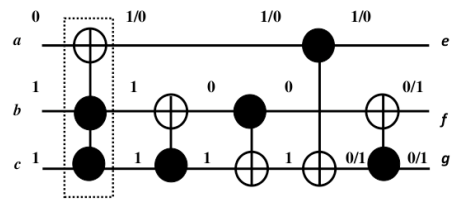

Fig. 4: Single Missing Gate Fault (SMGF)

III. Fault Detection In a Reversible Circuit

In this paper, we propose the test set generation technique for detecting all $SMGFs$ with the help of boolean expression based generator. The test set corresponding to each type missing gate fault is derived from the boolean expression.

A. Generation of Boolean expression of the fault free reversible circuit:

A reversible circuit is constructed using a number of reversible gates. If a circuit has N gates, then it has N levels starting from 0 to $(N-1)$. At each level of a reversible circuit, a reversible gate is placed. For a reversible circuit with n inputs, the input lines are numbered as $(i_0, i_1, i_2, \cdots, i_(n-1))$ starting from the top. Each input line is scanned from left to right to search a target, if a target is found then scan for the control lines for that gate and compute the output expression for that gate. This process is continued for all the gates of an input line. The final output expression corresponding to an input line is the output expression of the last target of that input line.

$Example$ 1: A reversible circuit with two input lines and four reversible gates is shown in Fig.5. At the input line a of the circuit, a reversible NOT gate is placed at level 0 and a 1-CNOT gate is placed at the level 1. Similarly, at the input line b, a reversible NOT gate is placed at level 2 and a 1-CNOT gate is placed at the level 3. The final output expression at line 0 is the output expression of the target at the level 1 of the circuit. Similarly, the final output expression at line 1 is the output expression of the target at the level 3 of the circuit. Table I shows the output of each line.

B. Generator for each missing gate in the circuit for Single Missing Gate Fault:

Let us assume a reversible circuit of size n (number of lines) and depth N (number of gates).

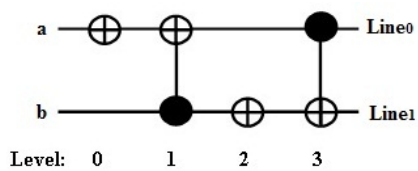

Fig. 5: Reversible circuit

TABLE I: Line number and corresponding Boolean expressions of all Toffoli gate of size 2

Line Number	Corresponding Boolean Expression
0	$((a \oplus 1) \oplus b)$
1	$((b \oplus 1) \oplus ((a \oplus 1) \oplus b)$

$Definition$ 1: The fault free expression at the jth line of an $(n \times n)$ reversible circuit denoted as the $OriginalExpression_j$ is the Boolean expression at the jth line of the circuit, $(0 \leq j \leq n-1)$. A reversible circuit having n number of lines consists of $OriginalExpression_0$, $OriginalExpression_1$,$\cdots$,$OriginalExpression_{n-1}$.

$Definition$ 2: When a gate is missing in the circuit, then the circuit is called a faulty circuit. Obviously, the Boolean expression obtained at any j^{th} line of the faulty circuit may differ from that obtained at the jth line of the fault free circuit. If the gate at the ith level of the circuit is missing, then the $FaultyExpression$ of the jth line of the reversible circuit denoted as the $FaultyExpression_{i,j}$ is the Boolean expression at the jth line for the fault at the ith gate, $(0 \leq i \leq N-1)$, $(0 \leq j \leq n-1)$. A reversible circuit with n lines produce $FaultyExpression_{i,0}$, $FaultyExpression_{i,1}$,$\cdots$$FaultyExpression_{i,n-1}$ for the SMGF fault at the ith gate.

The Boolean difference method [30] is an algebraic-type procedure for determining the complete set of tests that detects a given fault in the circuit for the stuck-at fault. We have introduced it for SMGF faults in reversible circuit.
$Definition$ 3: The Boolean difference $(\frac{dF_j}{dG_i})$ at the jth line due to the missing of the gate at the ith level of the circuit is defined as follows:
$\frac{dF_j}{dG_i} = F_j^o \oplus F_j^i$
where F_j^o is the $OriginalExpression_j$ and F_j^i is the $FaultyExpression_{i,j}$, $(0 \leq i \leq N-1)$, $(0 \leq j \leq n-1)$.

$Definition$ 4: The Boolean *generator* is the Boolean expression that is used to construct the test set to detect all the $SMGF$ fault of the circuit.

$Definition$ 5: The Boolean difference of the circuit with respect to fault of the gate at the ith level of the circuit denoted as $\frac{dF}{dG_i}$ is computed as follows:
$\frac{dF}{dG_i} = \Sigma\frac{dF_j}{dG_i} = (\frac{dF_0}{dG_i}) + (\frac{dF_1}{dG_i}) + \cdots\cdots + (\frac{dF_{n-1}}{dG_i})$, $(0 \leq i \leq N-1)$, $(0 \leq j \leq n-1)$
In this way, the generator of each gate is constructed and those generators are used to develop the Boolean generator of the circuit to construct the set of test patterns which can detect all possible Single Missing Gate Faults in the circuit.

Lemma 1: In an $(n \times n)$ reversible circuit having n lines and N gates, each Boolean difference $(\frac{dF}{dG_i})$ detects SMGF at the ith level.

Proof: In an (n × n) reversible circuit having n lines and N gates, the Boolean difference $(\frac{dF}{dG_i})$ which is used to detect the missing of ith gate from the circuit is computed as follows:

$$\frac{dF}{dG_i} = \Sigma\frac{dF_j}{dG_i}, (0 \leq i \leq N-1), (0 \leq j \leq n-1) \quad (1)$$

Σ represents the OR operation and

$$\frac{dF_j}{dG_i} = F_j^o \oplus F_j^i, (0 \leq i \leq N-1), (0 \leq j \leq n-1) \quad (2)$$

From the equation(1), it is observed that the possible value of $\frac{dF}{dG_i}$ is either the value '0' or some Boolean expression. From the equation (1), it is also observed that the value of $\frac{dF}{dG_i}$ is '0', only when the value of each of the $\frac{dF_j}{dGi}$ is '0' as *OR* operation is performed between two consecutive terms of $\frac{dF}{dG_i}$.
From the equation(2), we can see that the value of $\frac{dF_j}{dGi}$ is '0', if the term F_j^o and F_j^i are identical to each other and it is possible when the circuit is fault free. It means, if the value of $\frac{dF}{dG_i}$ is equal to '0', then the circuit is completely fault free.
If there is a $SMGF$ at any gate, obviously the output expression at any of the n lines would vary. It implies that there exists at least one line j for which the term F_j^o and F_j^i are not identical due to the fault in the circuit. It means $\frac{dF}{dG_i}$ can not be equal to '0' as OR operation is performed between two consecutive terms of $\frac{dF}{dG_i}$ and XOR operation is performed between the two terms of $\frac{dF_j}{dG_i}$. ■

C. Proposed Algorithm

In our proposed algorithm, the Boolean *generator* of the circuit is computed by function $B_{Gen} = \frac{dF}{dG_0}\Lambda\frac{dF}{dG_1}\Lambda \cdots \Lambda\frac{dF}{dG_{N-1}}$, where Λ denotes the logical AND operation and N is the number of gates of the circuit.
If $B_{Gen} \neq 0$, then the minimized form of the expression B_{Gen} is the Boolean *generator* of the circuit. If $B_{Gen} = 0$, then B_{Gen}^1 and B_{Gen}^2 are computed to calculate the Boolean *generator* of the circuit. These B_{Gen}^1 and B_{Gen}^2 are computed as follows:
Let $S = (s_o, s_1, \cdots, s_{N-1})$ be the set of all $\frac{dF}{dG_i}$, where $s_i = \frac{dF}{dG_i}$ for $(0 \leq i \leq N-1)$. Let $S_1 \subseteq S$, is the set containing maximum number of s_i's such that $B_{Gen}^1 = S_{i1}\Lambda S_{i2}\Lambda \cdots \Lambda S_{ik} \neq 0$, where $S_{ik} \in S_1$. B_{Gen}^2 is the rest of the s_i's of the set S. For each of the B_{Gen}^2, the $B_{Gen}^1 = 0$. The necessary term is taken from the B_{Gen}^2 to upgrade the B_{Gen}^1. After that, the B_{Gen}^1 is minimized to compute the Boolean *generator* of the circuit. The collection of the decimal value of the minterms of the *generator* is the test set to detect all SMGFs of the circuit. Our proposed algorithm is presented in Algorithm 1.

Algorithm 1: Boolean generator by Boolean difference method

Input : A reversible circuit of size n (no. of lines) with depth N consisting of N number of reversible k-CNOT gates g_0, g_1, $\cdots$, g_{N-1}.

Output: Boolean expression, which is the *generator* of the test set for detecting all possible Single Missing Gate Faults ($SMGF$) in the circuit.

begin

Step1: The original expression of the fault free circuit is calculated and stored in Original Expression string.

Step2: **for** i = *0 to* $(N-1)$ **do**
begin

2.1 Let us assume that a single missing-gate fault has occurred in the circuit RC at i^{th} level.

2.2 Compute the Faulty Expression of each line of the reversible circuit RC and store in the Faulty Expression.

2.3 Compute the Boolean difference of the circuit for the fault in $gate_i$ as follows:
$\frac{dF}{dG_i}$ = $\Sigma \frac{dF_j}{dG_i}$
$\frac{dF_j}{dG_i} = (OriginalExpression_j \oplus FaultyExpression_{i,j})$, j = 0 to $n-1$

Step3: Perform *AND* operation between each $\frac{dF}{dG_i}$ and store the output in the B_{Gen}.
B_{Gen} = $\frac{dF}{dG_0} \Lambda \frac{dF}{dG_1} \Lambda \cdots \Lambda \frac{dF}{dG_{N-1}}$,
Λ denotes the logical AND operation.

Step4: If the value of (B_{Gen} $\neq$0), then *generator* = B_{Gen}
GOTO Step 7
else GOTO Step 5

Step5: Initialize *count* = 0, *gen* = $\frac{dF}{dG_0}$,
Compute B^1_{Gen} and B^2_{Gen}.
for i = *1 to* $(N-1)$ **do**
begin

5.1 *gen* = *(gen) AND* ($\frac{dF}{dG_0}$)

5.2 if (*gen* $\neq$ 0) then
B^1_{Gen} = *gen*
else
B^2_{Gen} [count] = $\frac{dF}{dG_i}$
count = *count*+1

Step6: Compare the B^1_{Gen} with each B^2_{Gen} to upgrade the B^1_{Gen}, if necessary.
begin

6.1 if any term of B^2_{Gen} is a subset or completely matched with any term of B^1_{Gen}, do not upgrade the B^1_{Gen}.

6.2 if any term of B^2_{Gen} is not covered by any term of the B^1_{Gen}, then compare each term of the B^1_{Gen} with each term of the B^2_{Gen} and upgrade the B^1_{Gen} as follows:
B^1_{Gen} = B^1_{Gen} + highest matching term[i], i = 0 to (number of highest matching term - 1)
Repeat Step6.1 and Step6.2 between B^1_{Gen}*(s)* and other B^2_{Gen}*(s)*, if exist.

6.3 The minimized B^1_{Gen}*(s)* with lowest literals is the *generator* of the circuit.

Step7: Find out minterms of the *generator*. The decimal value of each minterm is the test pattern of circuit. The collection of decimal value of each minterm is the test set of the circuit.

$Example$ 2: Consider a ham3tc reversible benchmark circuit as ahown in Fig.6. In the circuit, the number of lines(n) is 3 and the number of gates(N) is 5. The output expressions are $OriginalExpression_0$; $OriginalExpression_1$ and $OriginalExpression_2$. The output expression of each line of the fault free reversible circuit is as follows: $OriginalExpression_0 = (a \oplus b \cdot c)$

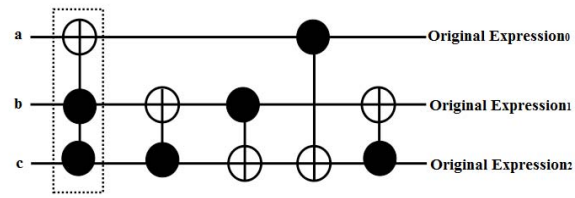

Fig. 6: ham3tc circuit (dotted box denotes removal of that gate)

$OriginalExpression_1 = ((b \oplus c) \oplus ((c \oplus (b \oplus c)) \oplus (a \oplus b \cdot c)))$
$OriginalExpression_2 = ((c \oplus (b \oplus c)) \oplus (a \oplus b \cdot c))$
Now, we inject a fault in the circuit by removing gate at $level0$. Removal of $gate_0$, results in change in the boolean expression of each line, which refers to the faulty expression. The Fig.7. shows that the presence of the fault in the circuit as $gate_0$ is missing. The faulty expression of each line is as follows:
$FaultyExpression_{0,0} = (a)$
$FaultyExpression_{0,1} = ((b \oplus c) \oplus ((c \oplus (b \oplus c) \oplus (a)))$
$FaultyExpression_{0,2} = ((c \oplus (b \oplus c)) \oplus (a))$
Similarly, we remove each gate one at a time and each time the faulty expression of each line is calculated as above. After finding out the faulty expression of each line for the missing of a gate and the fault free expression of each line, use the Step2 of the proposed algorithm to calculate the Boolean difference of the circuit for each gate as follows: For $gate_0$:

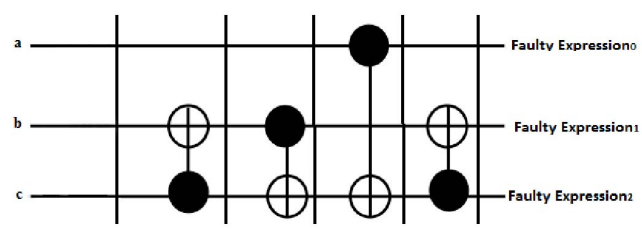

Fig. 7: Faulty Circuit Corresponding to ham3tc circuit

$\frac{dF}{dG_0} = (\frac{dF_0}{dG_0})$ + $(\frac{dF_1}{dG_0})$ + $(\frac{dF_2}{dG_0})$
$\frac{dF}{dG_0} = (OriginalExpression_0 \oplus FaultyExpression_{0,0})$ + $(OriginalExpression_1 \oplus FaultyExpression_{0,1})$ + $(OriginalExpression_2 \oplus FaultyExpression_{0,2})$
$\frac{dF}{dG_0} = ((a \oplus b \cdot c) \oplus a)$ + $(((b \oplus c) \oplus ((c \oplus (b \oplus c)) \oplus (a \oplus b \cdot c))) \oplus ((b \oplus c) \oplus ((c \oplus (b \oplus c) \oplus (a))))) + (((c \oplus (b \oplus c)) \oplus (a \oplus b \cdot c)) \oplus ((c \oplus (b \oplus c)) \oplus (a)))$
$\frac{dF}{dG_0} = bc$
Similarly, the Boolean difference of the circuit for all other gates is calculated as follows:
$\frac{dF}{dG_1} = c$, $\frac{dF}{dG_2} = b\bar{c} + \bar{b}c$

$\frac{dF}{dG_3} = \bar{a}bc + a\bar{b} + a\bar{c}$, $\frac{dF}{dG_4} = \bar{a}b\bar{c} + a\bar{b} + ac$
Use Step3 to calculate the B_{Gen}.
$B_{Gen} = (\frac{dF}{dG_0})AND(\frac{dF}{dG_1})AND(\frac{dF}{dG_2})AND(\frac{dF}{dG_3})AND(\frac{dF}{dG_4})$
Put the value of $\frac{dF}{dG_0}$, $\frac{dF}{dG_1}$, $\frac{dF}{dG_2}$, $\frac{dF}{dG_3}$, $\frac{dF}{dG_4}$ in the above equation to calculate the value of the variable B_{Gen}. As the value of the variable B_{Gen} would be 0, according to the Step4 of the algorithm, we have to go to Step5 to find out the *generator* of the circuit. In Step5, the variable *count* is initialized to 0 and the variable *gen* is initialized to $\frac{dF}{dG_0}$. The B^1_{Gen} and B^2_{Gen} is also computed in Step5. For this example, the variable i is varying from 1 to 4 as in the above circuit the number of gate is $N = 5$.
First Iteration $(i = 1)$:
gen = *(gen) AND* $(\frac{dF}{dG_1})$ = (bc) *AND* (c) = bc (Here, *gen* = bc as *gen* is initialized to the value of $\frac{dF}{dG_0}$).
As the value of *gen* is not equal to 0, the value of *gen* is assigned to the variable B^1_{Gen}. So, B^1_{Gen} = *gen* = bc.
Second Iteration $(i = 2)$:
gen = *(gen) AND* $(\frac{dF}{dG_2})$
gen = (bc) *AND* $(b\bar{c} + \bar{b}c)$ = 0. As the value of *gen* is 0, the $B^2_{Gen}(0)$ is computed. So, the value of B^1_{Gen} will not be upgraded.
B^2_{Gen}*(0)* = $(b\bar{c} + \bar{b}c)$
Third Iteration $(i = 3)$:
gen = *(gen) AND* $(\frac{dF}{dG_3})$
gen = (bc) *AND* $(\bar{a}bc + a\bar{b} + a\bar{c}) = \bar{a}bc$. As the value of *gen* is not equal to 0, the variable B^1_{Gen} is upgraded to the value of the variable *gen*. So, B^1_{Gen} = *gen* = $\bar{a}bc$.
Fourth Iteration $(i = 4)$:
gen = *gen AND* $(\frac{dF}{dG_4})$
gen = $(\bar{a}bc)$ *AND* $(\bar{a}b\bar{c} + a\bar{b} + ac)$ = 0, As the value of the variable *gen* is 0, the B^2_{Gen}*(1)* is computed. So, the value of the variable B^1_{Gen} will not be upgraded.
B^2_{Gen}*(1)* = $(\bar{a}b\bar{c} + a\bar{b} + ac)$
Now, use Step6 of the proposed algorithm to compare B^1_{Gen} with B^2_{Gen}*(s)*. According to the Step6.2 of the proposed algorithm, the availability of multiple number of B^1_{Gen} is checked. The B^1_{Gen} $(\bar{a}bc)$, is compared with the first B^2_{Gen}*(0)* to count number of literal matching between different terms of B^1_{Gen} and B^2_{Gen}. In this example, the term $(\bar{a}bc)$ of B^1_{Gen} is compared with both terms of B^2_{Gen}*(0)*. The term $(\bar{a}bc)$ of B^1_{Gen} has only one literal matching with both the term of B^2_{Gen}. According to the Step6.2, there will be two B^1_{Gen}.
B^1_{Gen}*(0)* = $(\bar{a}bc + b\bar{c})$ and B^1_{Gen}*(1)* =$(\bar{a}bc + \bar{b}c)$
As there is another B^2_{Gen}, repeat the Step6 of the algorithm. Compare both the B^1_{Gen} with the B^2_{Gen}*(1)* to find out updated value of B^1_{Gen}*(0)* and B^1_{Gen}*(1)*. At first, the B^1_{Gen}*(0)* is compared with the B^2_{Gen}*(1)* to find out the literal matching between them. The highest matching is there between the term $(\bar{a}bc)$ of B^1_{Gen}*(0)* and the term $(\bar{a}b\bar{c})$ of B^2_{Gen}*(1)*. So, the B^1_{Gen}*(0)* is upgraded to B^1_{Gen}*(0)* = $(\bar{a}bc + b\bar{c} + \bar{a}b\bar{c})$.
B^1_{Gen}*(1)* is compared with the B^2_{Gen}*(1)* to find out the literals matching betwen them. The highest matching is there between the term $\bar{a}bc$ of B^1_{Gen}*(1)* and the term $\bar{a}b\bar{c}$ of B^2_{Gen}*(1)*. So, the B^1_{Gen}*(1)* is upgraded to B^1_{Gen}*(1)* = $(\bar{a}bc + \bar{b}c + \bar{a}b\bar{c})$.
Use Step6.3 of the algorithm to minimize the $B^1_{Gen}(0)$ and $B^1_{Gen}(1)$. After the minimization, the B^1_{Gen}*(0)* = $\bar{a}b + b\bar{c}$ and B^1_{Gen}*(1)* = $\bar{a}b + \bar{b}c$.
As B^1_{Gen}*(0)* and B^1_{Gen}*(1)*, both contain same number of literals, So there will be two *generator* to detect all the $SMGF$ fault of the circuit.
generator(0) = $(\bar{a}b + b\bar{c})$ and *generator(1)* = $(\bar{a}b + \bar{b}c)$
Using Step7 of the algorithm, the derivation of the test pattern and the test set from the first *generator* i.e. generator(0) is shown in the Fig.8.

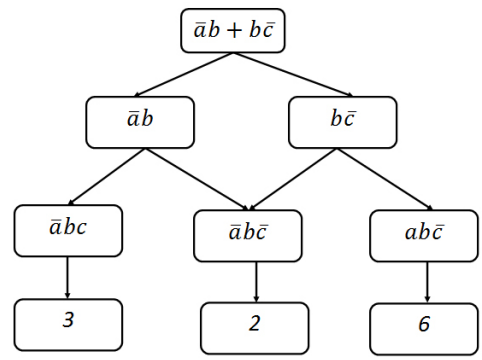

Fig. 8: Derivation of Test Set from generator(0)

The test set that is generated from the generator(0) is $\{Testpattern_0, Testpattern_1, Testpattern_2\}$ i.e. {3, 2, 6} i.e. {2, 3, 6}.
Derivation of the test pattern and the test set from the generator(1) is shown in the Fig.9.

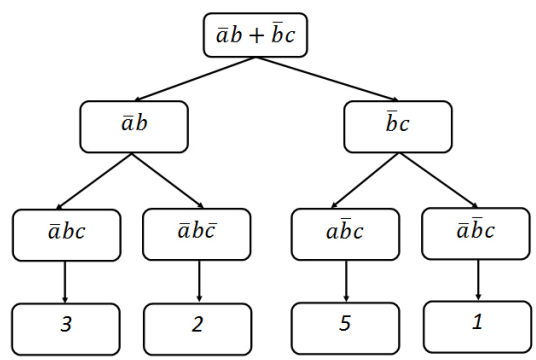

Fig. 9: Derivation of Test Set from generator(1)

The test set that is generated from the generator(1) is $\{Testpattern_0, Testpattern_1, Testpattern_2, Testpattern_3\}$ i.e. {3, 2, 5, 1} i.e. {1, 2, 3, 5}. As the test set generated from the generator(0) has less number of test pattern compared to the test set generated from the generator(1), so the test set {2, 3, 6} from the generator(0) will be used to detect the $SMGF$ fault of the circuit.

IV. EXPERIMENTAL RESULTS

We have experimented on several small and medium size reversible benchmark circuits[5] as shown in Table II. Column 2 shows the circuit name, column 3 and column 4 denote the number of gates (N) and the number of lines (n) of the circuit respectively. The boolean *generator* is shown in column 5 of table II. The set of test pattern that is generated from the *generator* is shown in column 6 of the table II.

TABLE II: Experimental Result

Serial No.	Circuit	N	n	Boolean generator of the circuit	Set of Test Pattern for SMGF
1	ham3tc	5	3	$(\bar{a}b + b\bar{c})$ **OR** $(\bar{a}b + \bar{b}c)$	{2,3,6} **OR** {1,2,3,5}
2	rd32	4	4	$(ab + \bar{a}bc)$ **OR** $(ab + a\bar{b}c)$	{6,7,12,13,14,15} **OR** {10,11,12,13,14,15}
3	xor5d1	4	5	$(a\bar{b}\bar{c}\bar{d})$	{16,17}
4	3-17tc	6	3	$(a\bar{b}c + ab\bar{c})$ **OR** $(a\bar{b}c + \bar{a}\bar{b}\bar{c})$	{5,6} **OR** {5,0}
5	mod5d1	8	5	$(abcd)$	{30,31}
6	4_49d3	12	4	$(abd + \bar{a}cd)$	{3,7,13,15}
7	hwb4d1	17	4	$(a\bar{b}cd + \bar{c}d + ac\bar{d})$	{1,5,9,10,11,13,14}
8	mod5d2	9	5	$(\bar{a}\bar{b}cd)$	{6,7}
9	rd32d1	4	4	$(ab + \bar{a}bc)$ **OR** $(ab + a\bar{b}c)$	{6,7,12,13,14,15} **OR** {10,11,12,13,14,15}
10	mod5d4	5	9	$(ab\bar{c})$ **OR** $(ab\bar{d})$ **OR** $(a\bar{c}\bar{d})$ **OR** $(b\bar{c}\bar{d})$	{24,25,26,27} **OR** {24,25,28,29} **OR** {16,17,24,25} **OR** {8,9,24,25}

V. CONCLUSION

This work has presented an algorithm for deriving the test sets for testing all single missing gate faults $(SMGFs)$ in the reversible circuit using Boolean generator. In this method, at first the Boolean generator is calculated using Boolean difference method and then the test set is generated from that generator. The traditional method of test set generation is not implemented here in this method. This is a completely new method to construct the generator which helps in generating the test set to detect all possible $SMGFs$ in the circuit.
We have a plan to enhance the algorithm for better scalability for larger circuits. Our future work includes the use of proposed algorithm for the $PMGFs$ and the $MMGFs$ in reversible circuits.

REFERENCES

[1] R.Landauer, "Irreversibility and heat generation in the computing process," *IBM journal of research and development*, vol. 5, no. 3, pp. 183–191, 1961.

[2] M.A.Nielsen and I.Chuang, "Quantum computation," *Quantum Information*, 2000.

[3] J. Mondal, D.K.Das, D.K.Kole, and H.Rahaman, "A design for testability technique for quantum reversible circuits," 10th *East-West Design and Test Symposium (EWDTS 2012)*, pp. 249–252, September 2012.

[4] D. Maslov, "Reversible logic synthesis," Ph.D. dissertation, University of New Brunswick, Canada, September 2003.

[5] D.Maslov, "Reversible logic synthesis benchmarks:www.cs.uvic.ca/dmaslov," 2004.

[6] P.Picton, "Optoelectronic, multivalued, conservative logic," *International Journal of Optical Computing*, vol. 2, pp. 19–29, 1991.

[7] R.C.Merkle, "Reversible electronic logic using switches," *Nanotechnology*, vol. 4, no. 1, pp. 21–40, 1993.

[8] R. Feynman, "Quantum mechanical computers," vol. 11(2), pp. 11–20, 1985.

[9] E.Fredkin and T.Toffoli, "Conservative logic," *International Journal of Theoritical Physics*, vol. 21, pp. 219–253, 1982.

[10] T. Toffoli, "Reversible computing," MIT Lab for Computer Science, Technical memo MIT/LCS/TM-151, February 1980.

[11] J. Mondal, D.K.Das, D.K.Kole, H.Rahaman, and B.B.Bhattacharya, "On designing testable reversible circuits using gate duplication," 17th *International Symposium on VLSI Design and Test (VDAT)*, pp. 322–329, 2013.

[12] J. P. Hayes, I. Polian, and B. Becker, "Testing for missing-gate faults in reversible circuits," *Asian Test Symposium (ATS 2004)*, pp. 100–105.

[13] D. P. Vasudevan, P. K. Lala, and J. P. Parkerson, "A novel approach for on-line testable reversible logic circuit design," *ATS 2004*, pp. 325–330.

[14] A.Chakraborty, "Synthesis of reversible circuits for testing with universal test set and c-testability of reversible iterative logic array," *VLSI Design*, pp. 163–168, 2008.

[15] H.Rahaman, D.Kole, D.K.Das, and B.B.Bhattacharya, "Optimum test set for bridging fault in a reversible circuit," *ATS 2007*, pp. 125–128, 2007.

[16] H. Rahaman, D. K. Kole, D. K. Das, and B. B. Bhattacharya, "Detection of bridging fault in reversible circuit," *International Symposium on VLSI Design and Test, VDAT2006*, pp. 384–392.

[17] D. K. Kole, H. Rahaman, D. K. Das, and B. B. Bhattacharya, "Derivation of optimal test set for detection multiple missing-gate faults in reversible circuits," 19th *IEEE Asian Test Symposium 2010 (ATS 2010)*, pp. 33–38.

[18] D.K.Kole, H.Rahaman, D.K.Das, and B.B.Bhattacharya, "Derivation of automatic test set for detection of missing gate faults in reversible circuit," *International Symposium on Electronic System Design (ISED 2011)*, pp. 200–205.

[19] D. K. Kole, H. Rahaman, D. K. Das, and B. B. Bhattacharya, "Detection of multiple missing-gate faults in reversible circuits," *International Workshop on Reversible Computation*, pp. 117–124, July 2010.

[20] D.K.Kole, H.Rahaman, D.K.Das, and B.B.Bhattacharya, "Derivation of test set for detecting multiple missing-gate faults in reversible circuit," *Journal of Computers and Electrical Engineering (Elsevier)*, vol. 39(2), pp. 225–236, February 2013.

[21] D. K. Kole, H. Rahaman, D. K. Das, and B. B. Bhattacharya, "Synthesis of online testable reversible circuit," 13th *IEEE International Symposium on Design and Diagnostics of Electronic Circuits and Systems*, pp. 277–280, 2010.

[22] K.Ramasamy, R.Tagare, E.Perkins, and M.Perkowski, "Fault localization in reversible circuits is easier than for classical circuits," *ATS*, 2004.

[23] I. Polian, J. P. Hayes, T. Fienn, and B. Becker, "A family of logical fault models for reversible circuits," *Asian Test Symposium*, pp. 422–427, 2005.

[24] S. Tayu, S. Fukuyama, and S. Ueno, "Universal test sets for reversible circuits." 16th *Annual International Computing and Combinatorics Conference (COCOON 2010)*, vol. 6196, pp. 348–357, 2010.

[25] K.N.Patel, J.P.Hayes, and I.L.Markov, "Fault testing of reversible circuit," *VTS*, pp. 410–416, 2003.

[26] H.Rahaman, D.Kole, D.K.Das, and B.B.Bhattacharya, "On the detection of missing gate faults in reversible circuits by a universal test set," *VLSI Design*, pp. 163–168, 2008.

[27] M. Zamani, M. B. Tahoori, and K. Chakrabarty, "Ping-pong test: Compact test vector generation for reversible circuits," 30th *IEEE VLSI Test Symposium (VTS)*, pp. 164–169, April 2012.

[28] P.K.Lala, *An Introduction to Logic Circuit Testing*. Morgan and Claypool Publishers, 1997.

[29] N.K.Jha and S.Gupta, *Testing of Digital System*. Cambridge University Press, 2003.

[30] Z. Kohavi, *Switching and Finite Automata Theory*, second edition ed. Tata McGraw- Hill, 1978.